Bridging Natural Language to Verified Implementation: A Neuro-Symbolic High-Assurance MBSE Pipeline in SysML v2

Amer Tahat, David Hardin, Isaac Amundson, and Darren Cofer
Collins Aerospace, An RTX Business
{amer.tahat, david.hardin, isaac.amundson, darren.cofer}@collins.com

***Abstract*—The development of high-confidence avionics systems demands rigorous allocation and end-to-end traceability from requirements to implementation, yet current Model-Based Systems Engineering (MBSE) environments provide little support for automated, mathematically verified allocation and traceability. While Generative AI (GenAI) offers a potential automation boon, its deployment in safety-critical workflows is severely constrained by hallucinations, prohibitive token costs, scarcity of open-source domain data, and restrictions on fine-tuning state-of-the-art proprietary models such as GPT-5.**

To address these barriers, we introduce a neuro-symbolic Spec–Code–Proof (SCP) copilot that integrates Codex-class agents into the DARPA PROVERS INSPECTA pipeline. The pipeline translates natural-language requirements into SysML v2 models with GUMBO contracts, generates HAMR code, and discharges Logika proofs, ensuring properties are mathematically preserved in the implementation. To enable this without fine-tuning, we introduce a *Meta-Rule* learning methodology that functions as a verifiable surrogate for weight adaptation. By crystallizing ephemeral in-context learning into persistent, expert-curated abstraction patterns, Meta-Rules drive a self-healing generate–verify–repair loop until verification constraints are formally satisfied within a given budget.

We frame our work as a specialized paradigm called Formal Specification-Driven Development (FSDD). FSDD elevates formal specifications—contracts, verification conditions, and machine-checkable proofs—to first-class artifacts that drive generation, verification, and repair. Consequently, FSDD establishes formal proofs, rather than tests, as the primary acceptance gate within a given budget.

We evaluate our methodology on safety-critical systems using an example sourced from an FAA requirements guidebook (>37 pages) relevant to avionics and cyber-physical domains. Our copilot achieved 100% code-level verification and a 58× increase in development task throughput compared to a baseline that, lacking Meta-Rules, failed to produce verifiable artifacts. This demonstrates a practical path to certification-grade automation under strict data and model governance constraints, reconciling the efficiency of Generative AI with the strict rigor required for certifiable critical system software.

***Index Terms*—SWE agents, Neuro-symbolic AI, Formal Methods, SysML v2, MBSE, Supervised Self-Healing, Supervised Meta-Rule Adaptation.**

I. Introduction

High-assurance avionics software development faces a persistent tension: system complexity continues to increase, while certification demands strong evidence that implementation faithfully realizes requirements and architectural intent. In practice, maintaining end-to-end traceability from natural-language requirements to architectural models and down to code remains costly and fragile. In many Model-Based Systems Engineering (MBSE) workflows, requirements allocation and refinement still involve manual, tool-fragmented steps that weaken (or sever) the link between the model-level assurance case and the software implementation. The resulting drift is often detected late—during integration or verification—driving rework, schedule risk, and certification friction [1].

Large Language Models (LLMs) appear well-suited to assist with this transition by translating requirements into structured specifications, contracts, and implementation scaffolding [2]. However, directly deploying LLMs in safety-critical workflows faces structural barriers. Beyond hallucinations and token-heavy prompting, aerospace environments face the *fine-tuning bottleneck*: domain-representative data is often scarce or proprietary, and fine-tuning frontier proprietary models (e.g., GPT-5-class systems [3]) is typically restricted by vendor policy or cost [2], [4].

In this setting, the key challenge therefore is not merely to generate plausible formal text, but rather the objective is to reach a *verified* artifact within bounded time, token, and human-review budgets. Recent theory on AI agents in verifiable settings provides a useful lens for this problem: past experience matters to the extent that it captures reusable algorithmic structure that reduces the time needed to solve new tasks [5]. Viewed through this lens, the important quantity is not raw fluency alone, but *cost per verified task*. That framing is particularly natural in MBSE pipelines where a checker is available and all candidate artifacts are filtered through hard acceptance gates.

As a result, these restrictions leave three capabilities—highly desirable in high-assurance certification-oriented development pipelines—unmet: (i) reducing token-heavy, brittle prompting; (ii) filtering or rejecting hallucinated behaviors with machine-checkable acceptance gates; and (iii) enabling non-formalists to work from English while still producing reusable, auditable evidence that persists across runs.

To address these barriers, we introduce a neuro-symbolic **Spec–Code–Proof (SCP)** *copilot*—a generate–verify–repair pipeline that turns natural-language requirements into traceable formal artifacts and verified implementations. SCP targets the three gaps above by (i) shifting effort from repeated token-

heavy prompting into reusable intermediate artifacts; (ii) using formal verification as the primary acceptance gate to catch hallucinated or underspecified behavior; and (iii) providing a low-entry English interface while accumulating governed, version-controlled specialization knowledge. Our approach integrates Codex-class software engineering agents into the DARPA PROVERS **INSPECTA** (Industrial-Scale Proof Engineering for Critical Trustworthy Applications) toolchain [1]. INSPECTA provides the formal backbone: **SysML v2** [6], [7] captures architecture, **GUMBO** [8], [9] expresses behavioral assume/guarantee contracts, **HAMR** [10], [11] generates executable memory-safe code, and **Logika** [11] discharges proof obligations using SMT solvers (e.g., Z3 [12] and CVC5 [13]). The intended outcome is an auditable workflow where the toolchain mechanically preserves properties established at the model level in the generated code. However, while this infrastructure ensures correctness *after* specification, the initial creation of formal models and contracts remains a manual, expertise-heavy bottleneck. Ideally, an agentic copilot would assist engineers in synthesizing these rigorous specifications from natural language, yet realizing this capability faces a critical barrier: effectively *specializing* an LLM to such a complex, verification-centric toolchain under base-model fine-tuning restrictions.

Recent mechanistic interpretability results suggest that In-Context Learning (ICL) can act like an "implicit fine-tuning" process during inference [14], but the effect is transient: once the context window is cleared, the learned behavior is lost. In high-assurance settings, this ephemerality is a practical obstacle: it forces repeated token-heavy prompting and makes it difficult to govern, audit, or reuse what the model "learned" across runs.

To stabilize this transient learning into a permanent asset, we introduce *Meta-Rules*: persistent, human-readable formalization abstractions that *crystallize* successful ICL interactions (derived from golden examples, tool feedback, and expert edits) into a reusable rule library. Meta-Rules serve as a verifiable surrogate for black-box weight adaptation: instead of modifying parameters, the system externalizes adaptation into version-controlled artifacts that are (i) inspectable by humans, (ii) exercised by the toolchain, and (iii) constrained by formal verification. In developer mode, the system iteratively proposes and refines Meta-Rules using semantic similarity metrics and HAMR/Logika feedback; only candidates that satisfy quantitative improvement criteria, pass verification, and receive human approval are retained. In user mode, the copilot injects this curated library into the context, effectively triggering the underlying implicit fine-tuning mechanism [14] to behave as a domain-adapted agent. This allows us to bypass the aforementioned fine-tuning restrictions while achieving comparable specialization, driving a generate–verify–repair loop until model integration and code-level verification succeed.

In addition to consistently producing practically formally verifiable output at low cost (Section VII)—effectively lowering the entry barrier for users—this method systematically accumulates a dataset of verified repair trajectories, alleviating current data scarcity and paving the way for future fine-tuning as model accessibility improves.

By anchoring the development workflow in these governed, externalized artifacts rather than opaque model behaviors, our approach aligns with the industry's emerging *Specification-Driven Development (SDD)* practices. Frameworks such as AWS Kiro [15] aim to curb "vibe coding" by structuring AI-assisted development around natural-language specifications and acceptance criteria rather than unconstrained prompt-driven generation. We extend this paradigm to *Formal Specification-Driven Development (FSDD)* by making *formal verification* the primary acceptance gate. In our workflow, specifications are refined into SysML v2 models and GUMBO contracts, then discharged through HAMR/Logika verification. This yields machine-checkable evidence alongside generated code, enabling a low-entry workflow that remains aligned with certification demands for traceability, repeatability, and auditable assurance artifacts.

In summary, the main contributions of this paper are:

1) **Trustworthy English-to-Implementation Pipeline:** A multi-agent SCP generate–verify–repair loop workflow integrated with INSPECTA (SysML v2 + GUMBO + HAMR/Logika) that generates verified contracts and code from requirements.
2) **Supervised Meta-Rules Self-Adaptation Algorithm:** A verifiable surrogate for fine-tuning that crystallizes ephemeral ICL behaviors into persistent Meta-Rules, enabling a governed generate–verify–repair loop driven by formal verification feedback and human approval.
3) **Benchmark Evaluation:** An evaluation of the Isolette benchmark [16] (a proxy for the avionics subsystems), demonstrating a substantial increase in verified performance per minute and complete verification success compared to a failing baseline.

The remainder of this paper is structured as follows. After establishing the SysML v2 GUMBO and HAMR/Logika background in Section II-A, Sections III and IV introduce our core methodology: the SCP architecture, the Meta-Rules framework, and the supervised self-adaptation algorithm. Section V compares Meta-Rules to fine-tuning and transient prompting, and discusses trade-offs. Section VI presents the Isolette running example. Section VII details the experimental evaluation and metrics. Section VIII discusses why Meta-Rules reduce cost. Section IX reviews related work. Section X discusses current limitations and future research directions, and Section XI concludes.

To foster community collaboration and accelerate future research, all toolchain source code, along with our curated dataset of successful repair trajectories, is released as open source.[1]

[1]https://github.com/loonwerks/INSPECTA-Spec-Code-Proof-Copilot

II. Background

A. INSPECTA High-Assurance System Development and Verification

The development of safety-critical systems, such as avionics, requires rigorous evidence that implementations faithfully realize their requirements. The DARPA PROVERS INSPECTA toolchain provides this verifiable setting by automatically integrating architectural design, code generation, and formal verification capabilities. Specifically,

- SysML v2 and GUMBO: Architecture models are captured in SysML v2, which natively supports a textual representation amenable to LLM processing. These models are augmented with GUMBO (Guarantees and Assumptions with Model-Based Operations) contracts. GUMBO allows engineers to attach formal specifications—including state variables, interface assumptions, and operational compute cases—directly to architectural components.
- HAMR and Logika: The HAMR framework analyzes these models to generate memory-safe executable skeleton code in languages like Slang/Scala (default and target of this paper), C, or Rust; mapping GUMBO assumptions and guarantees to formal verifiable requires and ensures blocks, respectively. Subsequently, in the case of Scala/Slang, the Logika formal verification framework uses SMT solvers (e.g., Z3, CVC5) to discharge proof obligations, ensuring the generated application code (manually or by LLM assistance) satisfies its GUMBO contracts across the entire state space.
- Finally, Rust and Verus: For Rust targets, HAMR generates Verus verification annotations. Verus allows developers to specify and prove properties using Rust's ownership model, mapping GUMBO assumptions and guarantees to Verus requires and ensures blocks, respectively. In this paper, our SCP copilot supports the default behavior.

A typical GUMBO block includes optional state variables, initialization guarantees, assumptions constraining inputs, global guarantees, and `compute_cases` clauses that align conditional requirements with case-specific guarantees (Listing 1).

Listing 1 provides an illustrative example.

```
language "GUMBO" /*{
  state lastCmd: On_Off;
  initialize
    guarantee initLastCmd: lastCmd == Off;

  compute
    assume lowerLEUpper: lower <= upper;
    guarantee lastCmdTracksOutput: lastCmd ==
        heat_control;

  compute_cases
    case REQ_1:
      assume mode == INIT;
      guarantee heat_control == Off;
    case REQ_2:
      assume mode == NORMAL && temp < lower;
      guarantee heat_control == On;
} */
```

Listing 1. Illustrative skeleton of a SysML v2 component with a GUMBO contract.

III. Neuro-Symbolic Contract Generation with Meta-Rules: An Overview

The SCP Codex Copilot introduces a multi-agent "Copilot Loop" that integrates Large Language Models (LLMs) with the INSPECTA toolchain.

A. SCP Copilot Neuro-Symbolic Workflow

The architecture operates as a coordinated multi-agent loop. As shown in Fig. 1, the system parses English requirements, applies Meta-Rules for formalization, and generates model contracts. It then validates these contracts using the *SysML v2 Sireum CLI* and *HAMR/Logika* tools. If verification fails, a "Judge Agent" uses minimal human hints to repair the specification before generating the final verified system code [17].

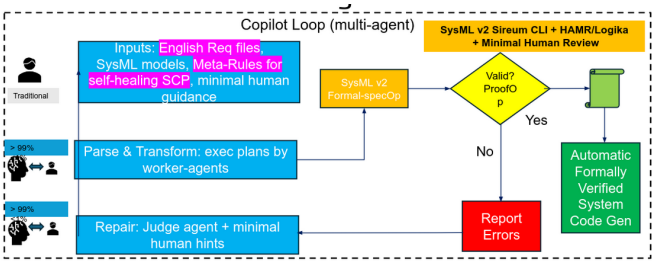

Fig. 1. SCP copilot neuro-symbolic generate–verify–repair loop. It parses English requirements, applies Meta-Rules, and utilizes the INSPECTA toolchain for verification.

B. Meta-Rules: Persistent, Auditable Formalization Abstractions

Meta-Rules are persistent, human-readable abstractions that constrain how the copilot translates natural-language requirements and architectural context into GUMBO contracts. Each Meta-Rule captures a reusable mapping from recurring requirement patterns (e.g., hysteresis, "no change" semantics, timeouts, latched modes) to concrete, tool-compatible GUMBO constructs (e.g., state variables, initialization guarantees, and structured `compute_cases`). In contrast to transient prompt-only conditioning, Meta-Rules are stored locally as version-controlled artifacts (a "rule book") that can be reviewed, curated, and audited by domain experts.

Remark 1. An intuitive proof-engineering analogy is helpful. When a proof engineer observes the same sequence of proof moves recurring across solved examples, the natural response is to package that pattern into a reusable lemma. Future proofs then become shorter because the engineer invokes the lemma rather than rediscovering the derivation from scratch. Solved requirement-to-contract episodes play the role of worked proof cases; a candidate Meta-Rule is the reusable lemma-like hypothesis extracted from them; semantic screening estimates where that hypothesis is likely to apply (a forecast model of its practical reusability to produce a practically verifiable specification); and the external verifier stack, together with human review, determines whether the hypothesis is worth retaining for a given budget. The analogy

captures the engineering intuition of our method: reusable formalization skill is learned from solved instances, represented symbolically, and validated by an external checker within a given budget.

C. Operational Modes of Use: User Mode vs. Developer Mode

Operationally, we distinguish two operational modes: **User Mode (application).** End users trigger the copilot with a low entry-point prompt that applies an existing, curated Meta-Rule library to translate requirements into candidate GUMBO contracts. The workflow then executes HAMR/Logika verification; failures trigger bounded, tool-guided repair until integration-level and code-level verification succeed.

Developer Mode (supervised self-adaptation learning). Developers iteratively refine the Meta-Rule library itself. Adaptation is supervised and gated: candidate Meta-Rule modifications are treated as hypotheses and retained only if they improve measurable criteria, pass verification, and receive explicit human approval. This yields cumulative improvement without explicit base-model fine-tuning.

1) Geometric Intuition: Practically Verified Semantic Cone: Figure 5 provides an intuition for why Meta-Rules act as a verifiable surrogate for fine-tuning in SCP. After injecting a curated Meta-Rule candidate $\mathcal{R}$, verification-aligned performance forms a local maximum over the base model manifold $\mathcal{M}$, centered around a golden example direction $\mathbf{g}$. The Meta-Rule *range of effect* is modeled as an angular neighborhood ($\pm\varepsilon_{\max}$): increasing ε expands the class of requirements and GUMBO contract patterns that can be produced from a small number of exemplars, while remaining within the accepted (budget-verifiable) region. This behavior is reflected in our evaluation (Sections VI and VII), where one golden GUMBO block can be leveraged—via Meta-Rule refinement and verification-guided repair—to synthesize multiple additional verified GUMBO blocks (e.g., one-to-six). An extended discussion of the geometry and budgeted acceptance interpretation is provided in Appendix A.

2) Example Meta-Rule: Listing 2 shows an excerpt from the rule book used in our evaluation. The rule instructs the agent to introduce explicit state when the requirements imply memory (e.g., hysteresis bands, "hold previous" semantics, time-dependent predicates). Such rules were motivated by baseline failures in which the agent attempted incompatible idioms (e.g., AADL pre-state operators) and could not consistently realize SysML v2 GUMBO state syntax.

```
/* * 2) WHEN TO USE EACH SECTION
 * 2.1 state
 * Add if the English requires memory:
 * - Hysteresis "shall not be changed" / "hold
     previous"
 * (e.g., Heat within desired band; Alarm no-change
     band).
 * - Timeouts (duration in mode > 1.0 s) if not
     directly
 * available from the platform as a primitive. */
// Pattern examples provided to the Agent:
state
  lastHeat: Isolette_Data_Model::On_Off;
  initElapsed: Base_Types::S64; // time units
```

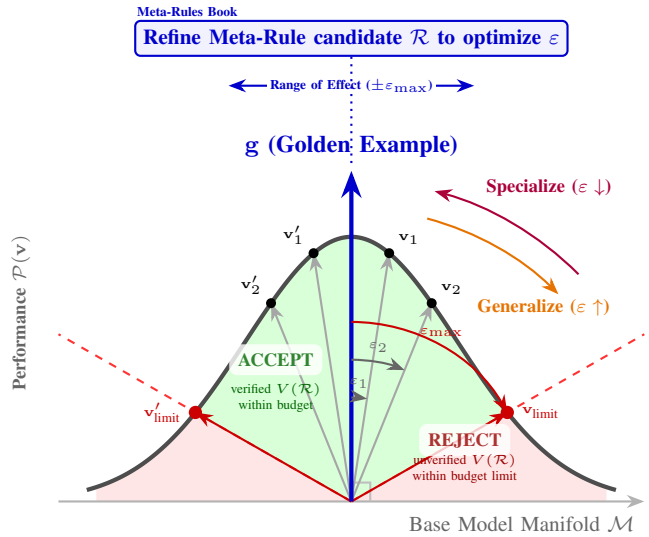

Fig. 2. Verified Semantic Cone with a bell-shaped verification-aligned performance landscape $\mathcal{P}(\mathbf{v})$ for supervised Meta-Rule refinement. The golden example direction $\mathbf{g}$ (blue) is perpendicular to the base model manifold $\mathcal{M}$. Under a Meta-Rule candidate $\mathcal{R}$, generated candidates (rays) intersect $\mathcal{P}$ at outcome points $\mathbf{v}_i$. Clockwise increases in ε promote generalization ($\varepsilon \uparrow$), while counterclockwise decreases promote specialization ($\varepsilon \downarrow$). The range of effect ($\pm\varepsilon_{\max}$) and boundary points $\mathbf{v}_{\text{limit}}$ separate the accepted region (green, verified $V(\mathcal{R})$ within budget) from the rejected region (red, unverified within the budget limit). Increasing ε expands the neighborhood in which $\mathcal{R}$ yields semantically correct, practically verifiable formalizations, enabling a small number of golden examples (e.g., one GUMBO block) to generalize to a larger class of verified contracts through semantic (not merely syntactic) refinement.

Listing 2. Extract from Gumbo FSE Agent Plan: Meta-Rules for State Formalization.

IV. Supervised Meta-Rule Self-Adaptation (Learning) and Acceptance Criteria

Meta-Rule adaptation is driven by three complementary gates:

(1) Semantic improvement. When golden reference formalizations are available (e.g., from validated contract blocks), we compute embedding-based cosine distance between the candidate contract text and the golden reference. Candidate Meta-Rule changes must demonstrate monotonic improvement against this semantic metric over a validation subset.

(2) Formal verification. All retained candidates must pass the INSPECTA verification workflow: SysML integration-level Logika verification followed by code-level Logika verification of HAMR-generated artifacts.

(3) Human governance. A human reviewer approves any Meta-Rule change before it is persisted into the rule book, ensuring the retained rule remains interpretable and transferable.

A. Practical Considerations

Meta-Rules are stored as local, version-controlled artifacts (a rule book) and are supplied to the agent at execution time together with the verification plan. This design keeps adaptation auditable and allows changes to be reviewed, reproduced,

and rolled back. We defer discussion of limitations and trade-offs to Section X.

B. The algorithm:

Algorithm 1 introduces the algorithm as a dual-phase neuro-symbolic process. Let R denote the target set of natural-language requirements, and let $\text{SAMPLE}(R) \subset R$ denote the sampled training subset used in developer mode during Phase I—in our setting, its size was 1/6 of R. Let $G = \{(r_g, g_g)\}$ denote the set of golden examples, where r_g is a golden requirement and g_g is its verified formalized GUMBO contract. Let M denote the retained Meta-Rule library, C the set of verified contracts, and F the set of flagged failures.

Phase I: Meta-Rule extraction, sample-based training, and predictive ϵ-learning. For each golden pair $(r_g, g_g) \in G$, EXTRACTRULE derives a candidate Meta-Rule m. For each sampled requirement $r \in \text{SAMPLE}(R)$, APPLYRULE uses m to produce a candidate formalized contract $\hat{g}$. The quantity $\text{DISTANCE}(\hat{g}, g_g)$ denotes a *semantic* distance between the generated formalized contract and the golden formalized contract, rather than a literal textual comparison. The threshold ϵ therefore controls the admissible degree of abstraction away from the golden exemplar. The objective is not to drive ϵ toward zero: if ϵ is too small, the rule becomes over-specialized and loses reusability; if ϵ is too large, the rule admits overly broad generations that require excessive repair or fail to verify within budget B. Accordingly, in developer mode ϵ is tuned across repeated calibration attempts to find a satisfactory intermediate value that best balances rule reuse and bounded repair. Verification is performed by $V(\hat{g}, B)$, where B is the repair budget in time and tokens for full repair cycles; the verifier returns a Boolean result ok together with feedback fb. While $\text{REPAIRBUDGETREMAINING}(B)$ holds, $\text{REPAIR}(\hat{g}, fb, B)$ revises the candidate using verifier feedback. The counter $okCount$ records how many sampled requirements are successfully verified for the candidate rule m, and τ is the minimum acceptance threshold required to retain m in M. If performance is unsatisfactory, $\text{DISCARDORREFINEWITHHUMAN}(m)$ permits human refinement of the rule, of ϵ, or both, and the developer-mode loop repeats.

Phase II: held-out validation and rule-guided formalization with bounded repair. After Phase I, the retained pair (M, ϵ) is evaluated beyond the sampled training subset. In developer mode, the Phase II loop is instantiated on requirements not used in $\text{SAMPLE}(R)$, testing whether the learned Meta-Rules and calibrated threshold ϵ generalize to unseen requirements while remaining repairable within budget B. In user mode, the same rule-guided synthesis procedure is applied to the full deployment set R. For each requirement r, $\text{SYNTHESIZEWITHRULES}(r, M)$ produces a candidate contract $\hat{g}$, which is checked again by $V(\hat{g}, B)$ and, if needed, revised by $\text{REPAIR}(\hat{g}, fb, B)$ while budget remains. If verification succeeds, the pair $(r, \hat{g})$ is added to C; otherwise the tuple $(r, \hat{g}, fb)$ is added to F. Operationally, the developer-mode loop (Phase I and Phase II) repeats until a satisfactory ϵ is found, namely one for which the retained Meta-Rule library remains reusable on unseen requirements within the available repair budget.

Algorithm 1: SCP Meta-Rules Learning, Application, and Verification-Guided Repair

Input: Requirements set R; golden examples G (English + formalized GUMBO); verifier V (HAMR/Logika); budgets B (time/tokens/repair cycles); similarity threshold ϵ; acceptance threshold τ.

Output: Verified contracts C; retained Meta-Rules library M; flagged failures F.

Initialization:

$M \leftarrow \emptyset$ `// persistent rule memory`

$C \leftarrow \emptyset$; $F \leftarrow \emptyset$

Phase I: Meta-Rule extraction and validation

foreach $(r_g, g_g) \in G$ **do**
  $m \leftarrow \text{EXTRACTRULE}(r_g, g_g)$
  $okCount \leftarrow 0$
  foreach $r \in \text{SAMPLE}(R)$ **do**
    $\hat{g} \leftarrow \text{APPLYRULE}(m, r)$
    if $\text{DISTANCE}(\hat{g}, g_g) > \epsilon$ **then**
      continue
    $(ok, fb) \leftarrow V(\hat{g}, B)$
    while not ok ***and*** $\text{REPAIRBUDGETREMAINING}(B)$ **do**
      $\hat{g} \leftarrow \text{REPAIR}(\hat{g}, fb, B)$
      $(ok, fb) \leftarrow V(\hat{g}, B)$
    if ok **then**
      $okCount \leftarrow okCount + 1$
  if $okCount \geq \tau$ **then**
    $M \leftarrow M \cup \{m\}$ `// retain only if it verifies and generalizes`
  else
    $\text{DISCARDORREFINEWITHHUMAN}(m)$

Phase II: Rule-guided formalization with bounded repair

foreach $r \in R$ **do**
  $\hat{g} \leftarrow \text{SYNTHESIZEWITHRULES}(r, M)$
  $(ok, fb) \leftarrow V(\hat{g}, B)$
  while not ok ***and*** $\text{REPAIRBUDGETREMAINING}(B)$ **do**
    $\hat{g} \leftarrow \text{REPAIR}(\hat{g}, fb, B)$
    $(ok, fb) \leftarrow V(\hat{g}, B)$
  if ok **then**
    $C \leftarrow C \cup \{(r, \hat{g})\}$
  else
    $F \leftarrow F \cup \{(r, \hat{g}, fb)\}$

Quality assessment and logging:

Log timestamps, token usage, latency, and repair count; store M for reuse.

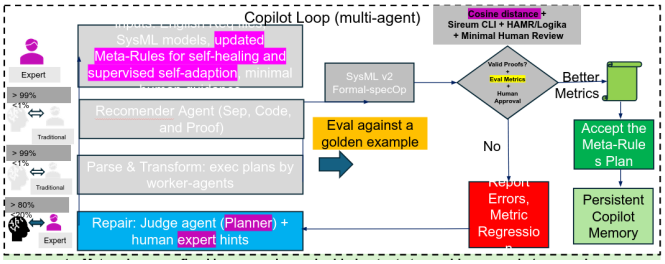

Fig. 3. Semi-automatic supervised loop used to learn and refine transferable Meta-Rules from requirement/GUMBO pairs under formal verification and human governance.

C. Discussion: Human-in-the-Loop Refinement

While Meta-Rule extraction and validation are highly automated, human oversight remains essential to ensure transferability and prevent drift. A critical aspect is the **generalization–specialization trade-off**, managed through semantic proximity checks, verification gates, and human governance. The threshold ϵ is not searched for as a theoretical maximum over all admissible semantic deviations. Rather, in developer mode it is calibrated as a user-satisfactory operating point that balances rule reusability against bounded repair cost. Accordingly, SCP adopts a satisficing criterion: ϵ is accepted once the learned pair (M, ϵ) achieves the user-required level of verification performance on unseen requirements within the available budget, rather than when ϵ is theoretically maximal. Human review remains essential in this calibration loop, since the final choice of ϵ reflects an engineering judgment about acceptable trade-offs among generality, verification success, repair effort, and cost.

V. Comparison to Fine-Tuning and Transient Prompting

This section situates Supervised Meta-Rules Adaptation relative to two common approaches for specializing LLMs: parameter fine-tuning and transient prompting / in-context learning (ICL). The comparison emphasizes suitability for high-assurance, certification-oriented workflows where auditability, governance, and reproducibility are first-class requirements.

A. Parameter Fine-Tuning

Fine-tuning modifies model weights to internalize domain-specific behavior. While it can yield strong task performance, it imposes significant practical barriers in aerospace settings: training data are often sensitive or scarce, compute costs can be prohibitive, and distributing tuned checkpoints may be restricted for proprietary frontier models. From an assurance perspective, fine-tuning produces opaque behavioral changes that are difficult to inspect, justify, or certify at the granularity expected in safety-critical development.

B. Transient Prompting and In-Context Learning

ICL adapts behavior via examples and instructions supplied in the prompt window. Mechanistic interpretability work suggests ICL can behave like "implicit fine-tuning" during inference [14], but its effects are ephemeral: the learned behavior disappears when context is reset. In practice, this ephemerality drives repeated long-context prompting, higher token cost, and limited governance over what was learned across sessions.

C. Supervised Meta-Rules Adaptation

Meta-Rules externalize learned formalization behavior into persistent, version-controlled artifacts. Rather than adapting weights, the system crystallizes successful ICL interactions into reusable abstraction patterns that are (i) inspectable by humans, (ii) constrained by formal verification gates (HAMR/Logika), and (iii) retained only under explicit human approval. This transforms adaptation into an auditable engineering process and provides a practical surrogate for fine-tuning under data, cost, and governance constraints.

In this paper, "surrogate for fine-tuning" is meant operationally rather than as a claim of universal equivalence to parameter fine-tuning. Meta-Rules provide persistent, reusable task specialization without modifying model parameters, while preserving auditability, rollback, and verification-gated acceptance. The surrogate claim is therefore about retained specialization behavior under governance constraints, not about matching fine-tuning on every task or benchmark.

Table I summarizes the adaptation mechanisms discussed in this section.

VI. A Running Example: The Isolette System

We use the Isolette Infant Incubator benchmark as a proxy for safety-critical subsystems [16]. The goal is to translate natural-language requirements into SysML v2 GUMBO contracts and then discharge both integration-level and code-level verification obligations using the SCP Copilot Self-healing mechanism with minimal user intervention.

A. Experimental Inputs and Outputs

Inputs. The experiment uses (i) natural-language requirements provided in a realistic FAA-style source document (37+ pages) containing plain text, tables, and figures, and (ii) SysML v2 models and shared libraries representing the Isolette architecture (e.g., `Monitor.sysml`, `Regulate.sysml`, plus SysML↔AADL shared libraries). These artifacts intentionally include realistic friction: domain terminology, cross-references, and requirements that imply memory (hysteresis/latched state) that must be encoded using SysML v2 GUMBO constructs accepted by the toolchain.

Outputs. The pipeline produces (i) SysML v2 models augmented with GUMBO contracts (including state/initialize/-compute_cases blocks), (ii) HAMR-generated implementation artifacts, and (iii) verification logs demonstrating successful discharge of Logika proof obligations for both model integration and code-level execution.

Experimental scope. In this study, the evaluated scope comprised the selected Monitor and Regulate threads, the associated GUMBO contract-bearing blocks and `compute_cases` generated or modified for those

threads, and the corresponding HAMR-generated proof-bearing methods examined by Logika (e.g., `initialize` and `timeTriggered`). Accordingly, later references to "complete integration-level and code-level verification" or "100% code-level verification" are with respect to this defined target scope.

B. Conditions: Baseline vs. Meta-Rule–Guided SCP

We compare two conditions:

Baseline (prompt-only ICL). A Codex-class agent is tasked to read the requirement document and directly generate correct SysML v2 GUMBO contracts without an external Meta-Rule library. This relies on transient in-context learning and ad-hoc prompt conditioning.

Meta-Rule–guided SCP. The same overall task is executed using a curated Meta-Rule library (Section III). The copilot is triggered with a low entry-point prompt, applies the rule book to guide formalization, and then executes a bounded generate–verify–repair loop driven by HAMR/Logika feedback. Importantly, the Meta-Rules used in this condition are treated as persistent engineering artifacts: they are created/refined in developer mode and retained only if they satisfy the acceptance gates described in Section III (semantic improvement where golden references exist, verification success, and human approval).

C. Moment 1: Baseline Failure (Token Explosion and Tool Mismatch)

In the initial evaluation, the baseline using Codex GPT-5.1 Max failed to encode requirements that implied memory (hysteresis, "no change") in SysML v2 GUMBO. Listing 3 shows the failure transcript: the agent repeatedly attempts incompatible pre-state idioms (e.g., `@pre`) and cannot converge on tool-accepted SysML v2 state syntax. This failure manifests as rapid token growth with no verifiable outcome.

Connection to the comparison. This is a concrete instance of the limitations discussed in Section V: prompt-only ICL is ephemeral and does not reliably internalize niche toolchain syntax, leading to long-context retries and token explosion. A standard remedy would be domain fine-tuning; however, fine-tuning frontier proprietary models was not available in our setting (vendor restrictions). This motivates a persistent, auditable specialization mechanism that does not require weight updates.

```
- Baseline - Codex worked for 33m with virtually no
    success

I started encoding the remaining FAA requirements (
    hysteresis, "no change", etc.) directly in the
    SysML GUMBO clauses. To do so I tried to
    introduce state variables such as last_alarm_cmd
     and last_heat_cmd so that we could talk about
    previous actuator commands (which is necessary
    for REQ-MA-3 and REQ-MHS-4). However the SysML
    parser rejects every variant of the state ...
    syntax I've tried, and it also treats @pre as an
     unsupported "classification test" operator. In
    other words, the current toolchain doesn't
    appear to accept any of the GUMBO constructs
    that refer to component state or pre-state
    values, which makes it impossible to capture the
     remaining requirements faithfully.

Could you share an example (or enable access to the
    original AADL GUMBO subclause) that shows the
    specific syntax this environment expects for
    declaring state variables and referencing their
    pre/post values? Once I know the accepted form I
     can finish adding the REQ-MA-3/5 and REQ-MHS-4
    cases.

Token usage: total=1,067,603 input=981,770 (+
    18,673,920 cached) output=85,833 (reasoning
    54,080)
To continue this session, run codex resume 019ab4a9-
    d677-7631-9b23-aeda551e9513
```

Listing 3. Baseline Failure Log: Transcript from the failed session showing parser rejection and massive token usage.

D. Moment 2: Success via Meta-Rules (Generate–Verify–Repair Under Gates)

Upon applying the curated Meta-Rules (e.g., Listing 2), the SCP copilot successfully formalized the requirements, inserted tool-accepted SysML v2 GUMBO blocks (including state for hysteresis), and completed both integration-level and code-level verification. Listing 4 summarizes the successful run.

Connection to the acceptance gates. The rules used here are not ad-hoc prompts; they are persistent assets curated under the developer-mode gates in Section III. In particular, Meta-Rule updates are retained only when they (i) improve semantic alignment where golden references exist (cosine-distance guidance), (ii) pass integration and code-level verification (hard gate), and (iii) receive human approval prior to being committed to the rule book. The user-mode run then applies these retained rules to reduce search and prevent token-heavy trial-and-error.

TABLE I
QUALITATIVE COMPARISON OF ADAPTATION MECHANISMS FOR HIGH-ASSURANCE PIPELINES.

	Fine-tuning	Prompting/ICL	Meta-Rules
Persistent across runs	Yes	No	Yes (external)
Requires weight updates	Yes	No	No
Auditability / review	Low	Low–Med	High
Governance / rollback	Low	Low	High
Runtime context overhead	Low	High	Med
Data/IP constraints	High	Low	Low
Verification integration	Indirect	Indirect	Direct

```
Codex optimized with meta-Rules. Worked for ~25
    minutes with complete success

- ...
- Regulate.sysml:281-353 adds the Manage Regulator
    Mode contract with state for the last mode and
    INIT timer plus helper predicates that encode
    the Regulator Status definition from Table A-10;
     cases REQ_MRM_1-REQ_MRM_4 reproduce the
    diagrammed transitions and latch semantics.
- Regulate.sysml:394-476 introduces the Manage Heat
    Source contract: Integration assumes the desired
    -range/temperature bounds, initialization forces
     the heat off, state remembers the previous
    actuation for hysteresis, and compute cases
    cover REQ_MHS_1-REQ_MHS_5 plus the explicit 6s
    latency references...

Verification
- ./sireum hamr sysml logika --sourcepath isolette/
    sysml
- isolette/hamr/slang/bin/run-logika.cmd
Both steps completed successfully (only the
    customary "Ignoring ... connection" notices);
    Logika proved every initialise and timeTriggered
     method for Monitor and Regulate threads.

Token usage: total=369,318 input=317,485 (+
    4,302,976 cached) output=51,833 (reasoning
    36,032)
```

Listing 4. Success Log: Transcript showing successful formalization and verification after applying Meta-Rules.

VII. Evaluation and Metrics

A. Experimental Settings

We evaluated SCP on the Isolette benchmark under two conditions: (i) a baseline prompt-only Codex agent and (ii) a Meta-Rule–guided SCP copilot. In both conditions, the agent was provided a realistic natural-language requirement source document [**?**] (37+ pages, containing figures, tables, and dense prose) along with the SysML v2 architecture models and shared libraries. The Meta-Rule–guided condition additionally supplied the curated Meta-Rule library and verification plan artifacts described in Section III. Success was defined as completion of integration-level and code-level verification using the INSPECTA workflow, culminating in a verified set of GUMBO-augmented SysML v2 models and verified HAMR-generated code. Unless otherwise noted, both conditions used the same base-model family, natural-language requirement source, SysML v2 architecture inputs, shared libraries, and INSPECTA verification workflow; the principal experimental difference was whether the curated Meta-Rule library and verification plan artifacts were supplied. Table II reports representative traces under this common setup.

B. Metrics

We report performance using token accounting, wall-clock time, and verification coverage over a defined set of proof-bearing units. Let U denote the set of targeted proof-bearing units within the experimental scope defined in Section VI. In this study, U comprises the selected contract-bearing artifacts and corresponding HAMR-generated proof-bearing methods checked by the INSPECTA workflow.

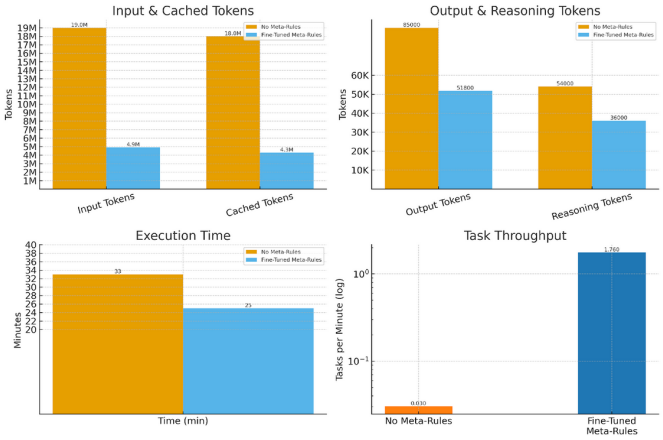

Fig. 4. Performance comparison: Before (Orange) vs. After Meta-Rules (Blue). The charts show a reduction in token usage and wall-clock time, converting a failing run into a fully verified run.

For a run r, we report wall-clock time $T_{\text{wall}}(r)$, uncached input tokens $N_{\text{in}}(r)$, cached input tokens $N_{\text{cache}}(r)$, reasoning tokens $N_{\text{reason}}(r)$, output tokens $N_{\text{out}}(r)$, and total tokens

$$N_{\text{total}}(r) = \big(N_{\text{in}}(r) + N_{\text{cache}}(r)\big) + N_{\text{out}}(r).$$

Let $U_{\text{verified}}(r) \subseteq U$ denote the subset of targeted units whose obligations are discharged by the end of run r. We then define

$$\rho(r) = \frac{|U_{\text{verified}}(r)|}{|U|},$$

as **verification coverage**,

$$\Theta(r) = \frac{|U_{\text{verified}}(r)|}{T_{\text{wall}}(r)},$$

as **verified throughput** (units/min), and

$$\kappa(r) = \frac{N_{\text{total}}(r)}{\max(1,\ |U_{\text{verified}}(r)|)},$$

as **token cost per verified unit**.

We also report an end-to-end success indicator $S(r) = 1$ iff $\rho(r) = 1$ and both integration-level and code-level verification complete over the targeted scope; otherwise $S(r) = 0$. Accordingly, "100% code-level verification" in this paper means full coverage of the targeted code-level proof-bearing units within the defined experimental scope.

C. Performance Comparison

The quantitative impact of Meta-Rules is visualized in Fig. 4. The orange bars represent the failing baseline, while the blue bars represent the Meta-Rule–guided approach.

As illustrated in Fig. 4:

- **Token efficiency:** The baseline consumed ≈19M cached input tokens and still failed. With Meta-Rules, cached input tokens dropped to ≈4.3M and uncached input dropped to ≈317k, while verification succeeded.
- **Reasoning efficiency:** Reasoning tokens decreased (e.g., from ≈54k to ≈36k in the representative logs), consistent with reduced search and fewer failed retries.

- **Wall-clock time:** Total runtime decreased from 33 minutes (failure) to ≈25 minutes (success).
- **Verified throughput (Θ):** Over the targeted proof-bearing units defined above, the Meta-Rule–guided approach achieved a reported $58\times$ improvement in verified throughput, i.e., $\Theta_{\mathrm{MR}}/\Theta_{\mathrm{Baseline}} = 58$, while also reaching end-to-end success. Table II reports the corresponding scope and metric fields ($|U|$, $|U_{\mathrm{verified}}|$, ρ, Θ, and κ) for both conditions. [17]

D. Representative Results

Table II reports representative logs from the baseline and Meta-Rule-guided runs. The baseline failed to produce a verifier-accepted artifact despite substantially higher token expenditure, whereas the Meta-Rule-guided run completed both integration-level and code-level verification.

The quantitative impact of Meta-Rules is also visualized in Fig. 4. The orange bars represent the failing baseline, while the blue bars represent the Meta-Rule-guided workflow.

Several observations follow from Table II. First, Meta-Rules convert a run with effectively unbounded cost per verified task into one that completes formal verification. Second, the successful run uses about $4.2\times$ fewer total tokens and about $4.3\times$ fewer cached input tokens than the failing baseline. Third, reasoning tokens fall from 54,080 to 36,032, consistent with the claim that the rule-guided workflow reduces unproductive search. Finally, wall-clock time decreases from 33 minutes to about 25 minutes while achieving complete verification with the reported $58\times$ improvement in verified throughput (Θ), showing that reduced search cost is accompanied by a meaningful runtime gain rather than only by better final correctness.

VIII. Discussion: Why Meta-Rules Reduce Cost

The results are best understood through the interaction of learning and search. The prompt-only baseline repeatedly expends computation to rediscover tool-specific formalization structure within a single long-context session, yet still fails to reach verifier-accepted artifacts. By contrast, the Meta-Rule-guided workflow begins from persistent formalization knowledge that has already survived semantic checks, formal verification, and human review. The observed reduction in reasoning cost and elapsed time is therefore not merely a prompting improvement; it is the expected consequence of replacing repeated rediscovery with governed reuse.

In developer mode, the system acquires reusable algorithmic structure from prior successful formalization episodes. In user mode, that structure shortens the trajectory to a verified solution. The verifier is essential in both phases: it acts first as selection pressure on which Meta-Rules are worth retaining, and later as the acceptance gate that determines whether a candidate artifact is actually useful. This interpretation is consistent with the recent theoretical framework lens from [5], see Appendix A

By "certification-grade automation," we mean automation that produces reviewable, replayable, traceable, and machine-checkable assurance artifacts compatible with certification-oriented engineering workflows. We do not claim automated certification approval; rather, the contribution is a governed workflow that preserves traceability and generates evidence in a form suitable for certification-facing development practice.

The results also clarify why the "surrogate for fine-tuning" description is only partially sufficient. Meta-Rules do help in settings where direct weight adaptation is unavailable or undesirable, but their value is broader. They preserve specialization as explicit engineering knowledge. That externalization makes adaptation inspectable, portable across model versions, and compatible with governance practices such as review, version control, and rollback. These are not incidental operational benefits; in certification-oriented environments they are part of the scientific and engineering contribution.

At the same time, the scope of the claim should remain precise. Our results do not show that Meta-Rules universally dominate fine-tuning, nor do they prove that all baseline failures are caused by missing knowledge rather than poor search. What they do show is that, in a verification-centric MBSE workflow with evolving tool-specific semantics, sparse domain data, and governance constraints, externalized rule memory is a practical and effective specialization mechanism. It improves the copilot's reachable formalization behavior while preserving the auditability expected in high-assurance engineering.

More broadly, the results suggest that future progress in high-assurance copilots may depend less on unconstrained scaling of prompt-only reasoning and more on explicit mechanisms for capturing, validating, and reusing formalization knowledge under checkers. In that sense, Meta-Rules are not an alternative to formal methods; they are a way of making generative models work *inside* a formal-methods-centered development process.

IX. Related Work

Recently, generative AI and particularly LLMs have shown promising potential to improve explainability and guide automated formal verification. Early efforts in AI-assisted theorem proving demonstrated the viability of integrating neural models with rigorous proof assistants. Notable examples include OpenAI's GPT-f and their subsequent fine-tuning (FT) experiments, which achieved early evidence of success in Metamath theorem proving [18]. Other initiatives have successfully applied LLMs to proof repair, automated tactic generation, and theorem diagnosis across a variety of foundational formal systems, such as Isabelle/HOL [19] and Coq [20]. In the specific context of the Lean theorem prover, recent AI advancements by commercial AI start-up models such as Harmonic Arsitotole and Axiom's Prover have showcased the ability of neuro-symbolic models to solve some complex, open mathematical problems [21] and reach gold medal performance in mathOlympics benchmarks [22].

Beyond pure mathematics, efforts in code verification have also increasingly adopted neuro-symbolic approaches. Microsoft has extensively explored AI-assisted code verification

TABLE II
REPRESENTATIVE RUN METRICS FOR THE BASELINE AND META-RULE-GUIDED SCP CONDITIONS.

Metric	Baseline (prompt-only)	Meta-Rule-guided SCP
Verified artifact produced	No	Yes
T_{wall} (minutes)	33	≈ 25
N_{in}	981,770	317,485
N_{cache}	18,673,920	4,302,976
N_{out}	85,833	51,833
N_{reason}	54,080	36,032
N_{total}	19,741,523	4,672,294
$\lvert U \rvert$ (targeted proof-bearing units)	*[x count]*	*[insert count]*
$\lvert U_{\text{verified}} \rvert$ (verified units)	*[insert count]*	*[insert count]*
ρ (verification coverage)	*[insert]*	*[insert]*
Θ (verified throughput, units/min)	*[insert]*	*[insert]*
κ (token cost per verified unit)	*[insert]*	*[insert]*

leveraging their F* framework [23]. Similarly, AWS recently introduced an SMT-backed hallucination prevention tool for generative AI [24], offered as a web service, and Apple released GSM-Symbolic to evaluate the limits of LLMs in symbolic reasoning tasks [25].

Within the domain of systems engineering, formal methods are currently being applied to modern MBSE standards. The OMG SysML v2 [26] was designed to support formal semantics, and reasoning platforms like Imandra [27] have developed transpilation tools to verify SysML v2 constraints, supporting an AI-assisted framework at the model level provided as a commercialized closed-source web service [27]. Additionally, models such as the PA-1 Archie model [28] pave the way to bridge formal methods and physical-world modeling. However, these MBSE applications are fundamentally limited to the model level; they analyze physical world properties and state machines but currently lack mechanisms for end-to-end formally verified code generation. Closer to our architectural focus, Tahat et al. introduced *AGREE-Dog* [37], a neuro-symbolic copilot for the OSATE AADL [30] environment that automates explainable compositional reasoning and repair using the AGREE tool.

Our work proposes a significantly deeper and more comprehensive framework. Rather than stopping at model-level analysis or functioning solely as an isolated proof assistant or specification language, the integration of the INSPECTA pipeline in our SCP copilot provides an end-to-end formal verification toolchain. It bridges the entire gap from English natural-language requirements down to GUMBO-annotated SysML v2 models, and ultimately synthesizes verified application code. By targeting high-assurance deployment platforms like the seL4 microkernel [31] and leveraging verified compilation microkit [32], our approach ensures that the strict mathematical rigor established at the architectural level is mechanically preserved all the way down to the executable implementation.

X. LIMITATIONS AND FUTURE WORK

In this section we identify three key directions for future research to address our current limitations and scale the architecture:

A. Expanding Evaluation to Heterogeneous Systems

To validate the generalizability of our Meta-Rules beyond the Isolette benchmark, we intend to expand our evaluation to diverse cyber-physical components. A primary target is the formalization of verified security appliances, such as the seL4-based firewall architectures developed by DornerWorks [33]. Applying our pipeline to these systems will test the adaptability of our GUMBO generation logic to distinct security policies and packet-filtering domains, moving beyond the control-theoretic patterns of the Isolette.

B. Integrating Closed-Loop Repair Heuristics for Rust Application Code

Currently, our copilot was tested on Scala/Slang [34] core application logic. We plan to integrate repair heuristics inspired by VERISTRUCT, a framework recently developed at Stanford for generating and repairing verified Rust code [35]. By adapting VERISTRUCT's syntax-guided and verification-aware repair strategies—specifically targeting the Verus toolchain—we aim to extend our SCP self-healing loop to the full synthesis of verified application logic in Rust.

C. Scalability and Physically-Aware Specification

Current LLMs often lack grounding in the physical constraints of cyber-physical systems. Future work will explore generating SysML v2 specifications using multimodal models capable of reasoning about physical world dynamics (e.g., spatial constraints, physics-based failure modes). Additionally, we aim to migrate the current local architecture to a cloud-based infrastructure, using SDD frameworks such as AWS Kiro, to parallelize the verification workload. Finally, the symbolic capability of our pipeline will naturally expand with the maturity of the underlying formal tools; specifically, analogous to [36], [37], we plan to leverage ongoing advancements in GUMBO that support compositional reasoning [10], enabling the verification of larger, hierarchically complex avionics systems.

XI. CONCLUSION

Our work demonstrates that combining the Meta-Rules methodology with INSPECTA toolchain verifiers to extract

and reuse formalization knowledge can significantly outperform direct, unconstrained model querying in a verification-centric MBSE environment. This neuro-symbolic approach paves the way for generating formally verified system code at realistic scale, with substantial improvements in efficiency and token cost per verified task while preserving guardrails from the system model through implementation, as well as auditability and human governance. It also lays the foundation for future research extending this work to more complex systems and larger scale enterprise-grade copilots.

Acknowledgment

This work was funded by DARPA contract [insert complete contract number]. The views, opinions, and/or findings expressed are those of the authors and should not be interpreted as representing the official views or policies of DARPA, the U.S. Department of Defense, or the U.S. Government.

Appendix A
Geometric and Performance Interpretation of Meta-Rule

This appendix extends the geometric interpretation of the Meta-Rule methodology by introducing a verification-aligned performance landscape over the embedding space. The resulting view clarifies how semantic generalization, formal verification constraints, and computational budget jointly determine admissible generations in the SCP workflow.

A. Embedding-Space Abstraction

Let $\mathcal{M} \subset \mathbb{R}^d$ denote the latent embedding manifold induced by a pretrained base model. Let $\mathbf{g} \in \mathbb{R}^d$ represent the embedding of a *golden example*, corresponding to a formally verified transformation:

English requirement $\rightarrow$ SysML v2 + GUMBO $\rightarrow$ HAMR code

$\rightarrow$ Logika proof discharge.

For conceptual clarity, we depict the base model's prior generative tendency as a horizontal manifold, while the golden example $\mathbf{g}$ is shown as a perpendicular semantic direction. This orthogonality is interpretive: it emphasizes that the golden example represents a correctness-anchored direction relative to unconstrained generation.

Under a Meta-Rule set $\mathcal{R}_i$, the SCP copilot produces a candidate embedding

$$\mathbf{v}_i = f_{\theta,\mathcal{R}_i}(x), \tag{1}$$

where f_θ is the base model and x is the input requirement. The semantic deviation from the golden example is quantified by

$$\varepsilon_i = \angle(\mathbf{v}_i, \mathbf{g}). \tag{2}$$

The parameter ε_i reflects controlled abstraction away from the exemplar.

B. Practically Verified Semantic Cone and Budgeted Acceptance

Meta-Rules constrain the effective sampling region of the model. Geometrically, admissible candidates form a conical neighborhood around $\mathbf{g}$. Let

$$V(\mathbf{v}_i) \in \{0, 1\}$$

denote the outcome of integration-level and code-level Logika verification under a bounded computational budget and humain review (tokens, repair cycles, and time constraints). A candidate is accepted if and only if:

$$V(\mathbf{v}_i) = 1. \tag{3}$$

The admissible region defines a *Practically Verified Semantic Cone*: the maximal angular neighborhood around $\mathbf{g}$ within which artifacts remain formally verifiable under the design budget. Outside this cone lies a *rejection region*—generations that either (i) violate logical constraints or (ii) require repair effort exceeding the verification budget. Thus, rejection is not merely semantic divergence but *budget-constrained unverifiability*

C. Performance Landscape and Token-Accounted Outcomes

We further interpret the embedding space as supporting a scalar performance functional $\mathcal{P}(\mathbf{v})$ that captures verification-aligned utility (e.g., semantic alignment to the golden exemplar, syntactic/tool compatibility, and repair efficiency). In Fig. 5, $\mathcal{P}$ is depicted as a bell-shaped (local maximum) curve centered near the golden example direction. Each ray denotes a candidate generation direction $\mathbf{v}_i$; its intersection with the performance curve yields an observed outcome point (dot) whose associated token-accounting vector summarizes execution cost:

$$\mathbf{t}_i = \begin{bmatrix} N_{\text{in}} \\ N_{\text{cache}} \\ N_{\text{reason}} \\ N_{\text{total}} \end{bmatrix},$$

where N_{in} is uncached input tokens, N_{cache} is cached input tokens, N_{reason} is reasoning tokens, and N_{total} is total tokens.

The *acceptance region* is the high-performance basin enclosed by the cone and the performance curve, where candidates remain verifiable under the design budget. The *rejection region* is the region bounded by the lower cone edge, the baseline, and the performance curve; candidates falling into this region are practically formally unverifiable within the allocated verification/repair budget.

Appendix B
Meta-Rules Through the Lens of Transductive Learning Theory

1) Transductive Agentic Search as a Theoretical Lens: We interpret this workflow through the transductive account of reasoning agents developed by Achille and Soatto [5]. In that view, an AI agent is modeled as a stochastic dynamical system operating in a verifiable setting, where a checker or

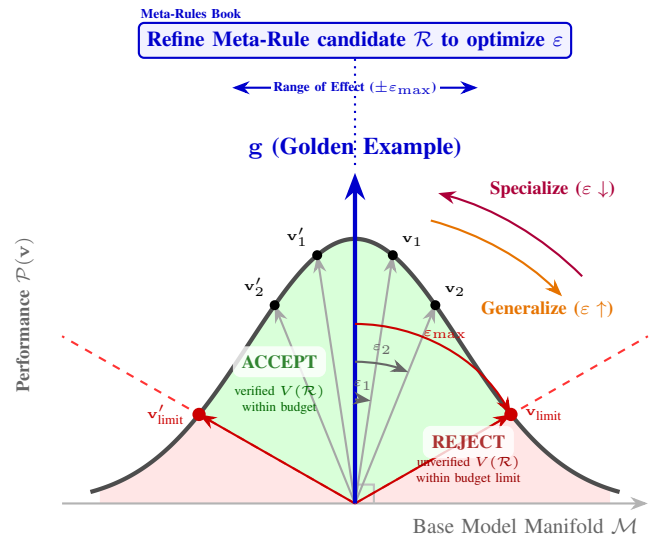

Fig. 5. Verified Semantic Cone with a bell-shaped verification-aligned performance landscape $\mathcal{P}(\mathbf{v})$ for supervised Meta-Rule refinement. The golden example direction $\mathbf{g}$ (blue) is perpendicular to the base model manifold $\mathcal{M}$. Under a Meta-Rule candidate $\mathcal{R}$, generated candidates (rays) intersect $\mathcal{P}$ at outcome points $\mathbf{v}_i$. Clockwise increases in ε promote generalization ($\varepsilon \uparrow$), while counterclockwise decreases promote specialization ($\varepsilon \downarrow$). The range of effect ($\pm\varepsilon_{\max}$) and boundary points $\mathbf{v}_{\text{limit}}$ separate the accepted region (green, verified $V(\mathcal{R})$ within budget) from the rejected region (red, unverified within the budget limit). Increasing ε expands the neighborhood in which $\mathcal{R}$ yields semantically correct, practically verifiable formalizations, enabling a small number of golden examples (e.g., one GUMBO block) to generalize to a larger class of verified contracts through semantic (not merely syntactic) refinement.

TABLE III
MAPPING TRANSDUCTIVE-SEARCH TERMINOLOGY TO THE SCP COPILOT.

Transductive construct	SCP interpretation
Stochastic dynamical system	LLM autoregressive generation with chain-of-thought and tool use
Verifiable oracle	INSPECTA: SysML v2 + GUMBO + HAMR/Logika
Proper time upper bound τ'	Tokens per wall-clock unit of time to reach a verified artifact
Reusable algorithmic structure	Persistent Meta-Rules distilled from golden examples, verifier feedback, and expert edits
External memory source	Curated Meta-Rule **library** used to condition search
Low-cost applicability control	Semantic proximity gate ϵ for rule injection
Savant regime	Prompt-only trial-and-error that consumes budget without acquiring reusable structure

reward function can determine whether a candidate solution is acceptable. The key objective is therefore not prediction error alone, but the computational effort required to reach a verified solution.

Table III maps the main transductive constructs to our SCP pipeline.

Definition 1 (Proper Time, adapted from [5]): Let v be a stochastic solver generating a reasoning trajectory h from input specification x and terminating in a verified artifact a. The *proper time* to reach a is

$$\tau_v(x \Rightarrow a) = \min_{h \Rightarrow a} \frac{T(h)}{v(h \mid \mathrm{enc}(x))}, \tag{4}$$

where $T(h)$ is the trajectory length and $v(h \mid \mathrm{enc}(x))$ is the probability that the solver generates a successful trajectory from the encoded input. In our setting, an upper bound of proper time can be operationally reflected by token usage per elapsed wall time required to obtain a verifier-accepted artifact.

The following three results provide the theoretical vocabulary we use throughout the paper.

Theorem 1 (Better compression implies faster search, adapted from [5]): If conditioning on a memory source M shortens the effective description of a successful trajectory h by Δ bits, then the expected discovery time improves by a factor of 2^{Δ}.

In our setting, M corresponds to the persistent Meta-Rule library. Intuitively, Meta-Rules help because they compress recurrent requirement-to-formalization patterns into reusable symbolic structure, making productive trajectories easier to reach than under prompt-only search.

Theorem 2 (Information is speed, adapted from [5]): Let $\tau_v(h)$ denote the proper time of a baseline stochastic solver and let $\tau_{v|M}(h)$ denote the proper time of the same solver conditioned on memory source M. Then the logarithm of the speed-up is

$$\log \frac{\tau_v(h)}{\tau_{v|M}(h)} = I_v(h : M), \tag{5}$$

where $I_v(h : M)$ is the algorithmic information shared between the successful trajectory and the conditioning memory.

Remark 2. We do not claim to measure algorithmic mutual information directly. Rather, in Section VII, we can interpret reductions in tokens, retries, and wall-clock time—together with the reported throughput improvement—as observable efficiency signals consistent with this transductive account.

Theorem 3 (Learning requires time optimization; otherwise search degenerates, adapted from [5]): If a solver has access to a verifier but is not penalized for the proper time it consumes, then exhaustive trial-and-error can be optimal, and reusable reasoning structure need not be learned. Conversely, any system that improves time-to-solution must acquire reusable reasoning information (Meta Rules) that reduces the search effort for future tasks.

This result explains the failure mode that motivates our architecture. In a verification-centric MBSE workflow, an unconstrained prompt-only agent can drift toward long, costly trial-and-error traces without acquiring persistent formalization skill. Our Meta-Rules are designed to prevent that behavior by externalizing reusable requirement-to-GUMBO structure in developer mode and then reusing it to bound search in user mode, see Section III.

An intuitive proof-engineering analogy is helpful. When a proof engineer observes the same sequence of proof moves recurring across solved examples, the natural response is to package that pattern into a reusable lemma. Future proofs then

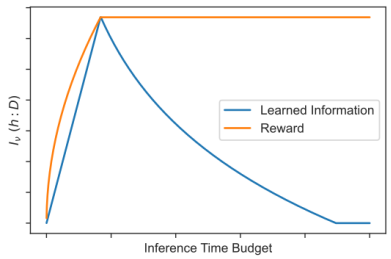

Fig. 6. Main result of [5]: Algorithmic structure ($I_v(h : D)$) learned at inference time from the in context data D initially grows and enables faster solutions in ICL setting, but beyond a threshold the system enters a "savant regime" where reward saturates and learned information decreases, as brute-force search replaces reusable reasoning.

become shorter because the engineer invokes the lemma rather than rediscovering the derivation from scratch. Taken together, these results motivate the SCP design adopted in this paper, Section III.

A. Meta-Rules Through the Lens of Transductive Learning

Let M denote a library of retained Meta-Rules, and let $m \in M$ be a rule with semantic centroid c_m in embedding space. For a new requirement r, with embedding $e(r)$, we define a semantic applicability test

$$d_m(r) = 1 - \cos(e(r), c_m), \qquad \text{apply } m \text{ only if } d_m(r) \leq \epsilon_m, \tag{6}$$

where ϵ_m is tuned offline in developer mode. The role of this gate is not to prove correctness; rather, it is to cheaply filter obviously mismatched rule applications before expensive verification is attempted. After rule injection, the copilot enters a bounded generate–verify–repair loop. Under the transductive lens, the resulting speed-up is expected to grow with the amount of reusable information the rule library shares with successful trajectories. We therefore interpret our empirical token and runtime reductions as *consistent with* the theory that reusable external memory can lower proper time by increasing the probability of productive search paths [5].

References

[1] D. Hardin, I. Amundson, J. Babar, D. Cofer, S. Hasan, K. Hoech, J. Belt, J. Hatcliff, Robby, and S. Hallerstede, "Automated sysml v2 system model to memory-safe language code generation for avionics applications," in *Proceedings of the IEEE/AIAA Digital Avionics Systems Conference (DASC)*, 2025, author version / preprint (uploaded as hardin2025dasc.pdf).

[2] OpenAI, "GPT-5.2 Codex System Card," https://openai.com/index/gpt-5-2-codex-system-card/, 2025, accessed: 2026-02-12.

[3] S. Bubeck, C. Coester, R. Eldan, T. Gowers, Y. T. Lee, A. Lupsasca, M. Sawhney, R. Scherrer, M. Sellke, B. K. Spears, D. Unutmaz, K. Weil, S. Yin, and N. Zhivotovskiy, "Early science acceleration experiments with gpt-5," 2025. [Online]. Available: https://arxiv.org/abs/2511.16072

[4] OpenAI, "GPT-5 System Card," https://openai.com/index/gpt-5-system-card/, 2025, accessed: 2026-02-12.

[5] A. Achille and S. Soatto, "Ai agents as universal task solvers," *Entropy*, vol. 28, no. 3, 2026. [Online]. Available: https://www.mdpi.com/1099-4300/28/3/332

[6] Object Management Group (OMG), "Sysml v2 model formalism working group," https://www.omgwiki.org/OMGSysML/doku.php?id=sysml-roadmap:sysml_v2_model_formalism_working_group, accessed: 2026-02-10.

[7] Object Management Group, "Omg system modeling language (sysml) v2 specification," 2025. [Online]. Available: https://www.omg.org/spec/SYSML

[8] J. Hatcliff, D. Stewart, J. Belt, Robby, and A. Schwerdfeger, "An AADL contract language supporting integrated model- and code-level verification," *Ada Letters*, vol. 42, no. 2, p. 45–54, April 2023. [Online]. Available: https://doi.org/10.1145/3591335.3591339

[9] Galois, "Gumbo: Grand unified modeling of behavioral operators," 2024. [Online]. Available: https://www.galois.com/project/gumbo

[10] J. Hatcliff, J. Belt, Robby, and T. Carpenter, "HAMR: An AADL multi-platform code generation toolset," in *10th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA)*, ser. LNCS, vol. 13036, 2021, pp. 274–295.

[11] *Sireum HAMR: High Assurance Modeling and Rapid Engineering for Embedded Systems*, Kansas State University, 2025. [Online]. Available: http://hamr.sireum.org/

[12] L. De Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in *Proceedings of the Theory and practice of software, 14th international conference on Tools and algorithms for the construction and analysis of systems*, ser. TACAS'08/ETAPS'08, 2008, pp. 337–340.

[13] C. Barrett, C. L. Conway, M. Deters, L. Hadarean, D. Jovanović, T. King, A. Reynolds, and C. Tinelli, "CVC4," in *Proceedings of the 23rd international conference on Computer aided verification*, ser. CAV'11, 2011, pp. 171–177.

[14] J. von Oswald, E. Niklasson, E. Randazzo, J. Sacramento, A. Mordvintsev, A. Zhmoginov, and M. Vladymyrov, "Transformers learn in-context by gradient descent," in *Proceedings of the 40th International Conference on Machine Learning*. PMLR, 2023, pp. 35 151–35 174.

[15] AWS News Blog, "Prevent factual errors from llm hallucinations with mathematically sound automated reasoning checks (preview)," AWS Blog Post, 2024, accessed: 2025-05-11.

[16] J. Hatcliff *et al.*, "The isolette system: Illustrating end-to-end artifacts for rigorous model-based engineering," in *International Symposium On Leveraging Applications of Formal Methods, Verification and Validation (ISoLA)*, 2024, author version (uploaded as Hatcliff-etal-ISOLA2024-Isolette-Overview.pdf).

[17] A. Tahat *et al.*, "Neurosymbolic automated contract and code generation for sysml v2 models," in *Collins Aerospace*, 2026, author version / preprint (uploaded as slides.pdf).

[18] S. Polu and I. Sutskever, "Generative language modeling for automated theorem proving," *arXiv preprint arXiv:2009.03393*, 2020.

[19] E. First, M. N. Rabe, T. Ringer, and Y. Brun, "Baldur: Whole-proof generation and repair with large language models," *arXiv preprint arXiv:2303.04910*, 2023.

[20] A. Tahat, D. Hardin, A. Petz, and P. Alexander, "Proof repair utilizing large language models: A case study on the copland remote attestation proofbase," in *Proceedings of International Symposium On Leveraging Applications of Formal Methods Verification and Validation (AISolA)*, 2024.

[21] Axiom, "Territory," https://axiommath.ai/territory, 2026, accessed: 2026-02-12. [Online]. Available: https://axiommath.ai/territory

[22] T. Achim, A. Best, A. Bietti *et al.*, "Aristotle: Imo-level automated theorem proving," arXiv:2510.01346, 2025. [Online]. Available: https://arxiv.org/abs/2510.01346

[23] N. Swamy, C. Hritcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P. Strub, M. Kohlweiss, J. K. Zinzindohoue, and S. Z. Béguelin, "Dependent types and multi-monadic effects in F," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, R. Bodík and R. Majumdar, Eds. ACM, 2016, pp. 256–270. [Online]. Available: https://doi.org/10.1145/2837614.2837655

[24] Amazon Web Services, "Automated reasoning checks in amazon bedrock guardrails," 2025. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html

[25] S. I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar, "GSM-symbolic: Understanding the limitations of mathematical reasoning in large language models," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=AjXkRZIvjB

[26] *OMG Systems Modeling Language (SysML), Version 2.0 Beta 2*, Object Management Group, February 2024. [Online]. Available: https://www.omg.org/spec/SysML/2.0/Beta2/Language/PDF

[27] Imandra, "Imandra sysml: Formal verification for sysml v2 models," 2024. [Online]. Available: https://www.imandra.ai/sysml

[28] S. Neema, S. Jha, A. Nagel, E. Lew, C. Sureshkumar, A. Gordic, C. Shimmin, H. Nguygen, and P. Eremenko, "On the evaluation of engineering artificial general intelligence," 2025. [Online]. Available: https://arxiv.org/abs/2505.10653

[29] A. Tahat, D. Hardin, A. Petz, and P. Alexander, "Metrics for large language model generated proofs in a high-assurance application domain," in *High Confidence Software and Systems Conference (HCSS'24)*, 2024.

[30] P. H. Feiler and D. P. Gluch, *Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis & Design Language*, 1st ed. Addison-Wesley Professional, 2012.

[31] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood, "sel4: Formal verification of an os kernel," in *Proceedings of the 22nd ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, 2009. [Online]. Available: https://doi.org/10.1145/1629575.1629596

[32] I. Velickovic, *Microkit User Manual*, 2025. [Online]. Available: https://github.com/seL4/microkit/blob/main/docs/manual.md

[33] J. Hatcliff, "Model-based development for seL4 Microkit/Rust with integrated formal methods using HAMR," Presentation at the seL4 Summit 2025, 2025, accessed: 2025-05-19. [Online]. Available: https://sel4summit2025.sched.com/event/26GD3

[34] Robby and J. Hatcliff, "Slang: The Sireum programming language," in *10th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA)*, ser. LNCS, vol. 13036, 2021, pp. 253–273.

[35] C. Sun, Y. Sun, D. Amrollahi, E. Zhang, S. Lahiri, S. Lu, D. Dill, and C. Barrett, "VeriStruct: Ai-assisted automated verification of data-structure modules in verus," 2025. [Online]. Available: https://arxiv.org/abs/2510.25015

[36] D. Cofer, A. Gacek, S. Miller, M. W. Whalen, B. LaValley, and L. Sha, "Compositional verification of architectural models," in *NASA Formal Methods*, A. E. Goodloe and S. Person, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 126–140.

[37] A. Tahat, I. Amundson, D. Hardin, and D. Cofer, "Agree-dog copilot: A neuro-symbolic approach to enhanced model-based systems engineering," in *Bridging the Gap Between AI and Reality (AISoLA 2025), Selected Papers*, 2025. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-032-07132-3_8